

Module 05 — Sources and the Medallion Architecture

Tier: ● Beginner · Duration: 90 min · Prerequisites: Module 04

Change from original plan: This module was previously 60 minutes. It's now 90 minutes because the medallion architecture reliably generates significant questions from newcomers, and source freshness was getting rushed. Both topics now have adequate time.

Agenda

Time	Duration	Topic	Learning Goal	Mode	Participant Activity	Materials	Trainer Notes	Checkpoint
00:00	10 min	Recap Module 04	Confirm materialization mental model	Q&A	Answer from memory	—	Ask all 4 prep questions. "What SQL does a table materialization generate?" — expect DROP + CREATE TABLE AS SELECT	All 4 correct
00:10	20 min	The medallion architecture	Know what each layer owns, who writes to it, and why	Present + diagram	Annotate layer diagram	Whiteboard	Draw it by hand, not from a slide. Emphasise: dbt does NOT own Bronze. Lambda does. dbt starts at Staging.	"Who writes to the Bronze layer?"
00:30	15 min	<code>sources.yml</code> — declaring sources	Understand how dbt knows about Bronze tables	Present + live file	Follow along in editor	<code>sources.yml</code> from repo	Write a minimal <code>sources.yml</code> live. Key point: without this declaration, <code>{{ source() }}</code> fails — dbt doesn't auto-discover tables.	"What happens if you call <code>{{ source() }}</code> without a <code>sources.yml</code> declaration?"

Time	Duration	Topic	Learning Goal	Mode	Participant Activity	Materials	Trainer Notes	Checkpoint
00:45	10 min	<code>{{ source() }}</code> vs hardcoding	Know why <code>source()</code> is mandatory and hardcoding is forbidden	Present + live compile	Compare compiled outputs	VS Code + <code>dbt compile</code>	Run <code>dbt compile</code> on a model using <code>source()</code> and one using a hardcoded name. Show the DAG difference in <code>dbt docs serve</code> .	"Name two things you lose when you hardcode a table name instead of using <code>source()</code> "
00:55	15 min	Source freshness	Know how to declare and check freshness	Present + live demo	Follow along	This doc	Run <code>dbt source freshness</code> live. Show the output. Explain where this fits in Airflow: freshness check before models run.	"What does dbt do if a source is stale and freshness is set to <code>error</code> ?"
01:10	25 min	Exercise: write a staging model from scratch	Write a staging model that correctly references a source	Practice	Solo exercise	Exercise below	Circulate. Most common mistakes: missing <code>sources.yml</code> declaration, hardcoded table reference, wrong Jinja delimiter.	Exercise complete, <code>dbt compile</code> succeeds
01:35	10 min	Debrief + prep questions	Consolidate	Debrief	Verbal	—	Ask: "if Bronze data stops arriving from HubSpot, how does dbt know?" — answer: source freshness check.	—

Content

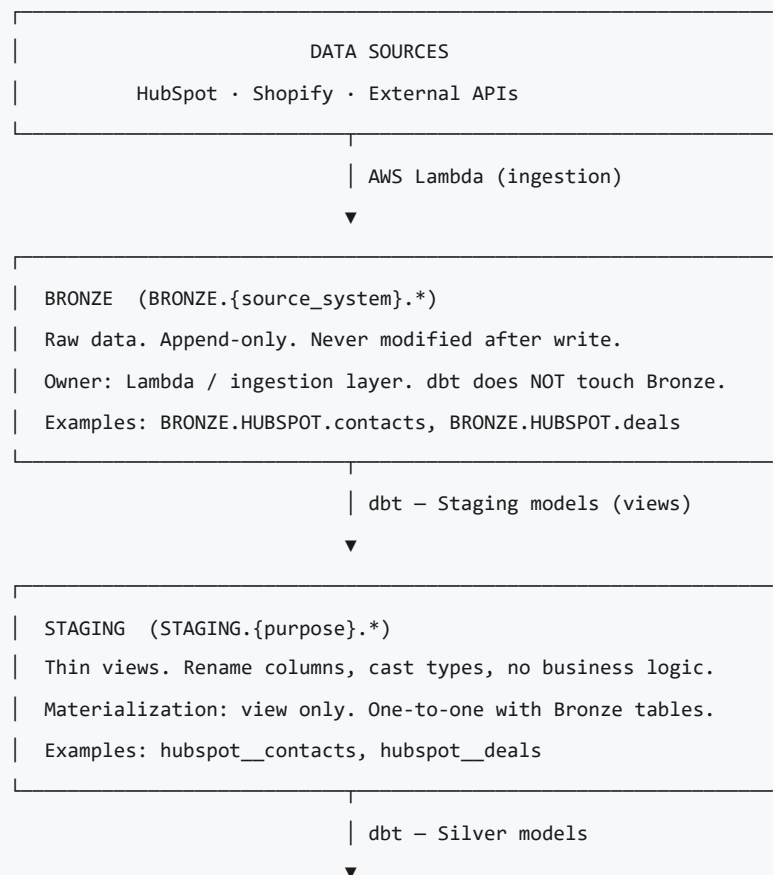
Part A — The Medallion Architecture

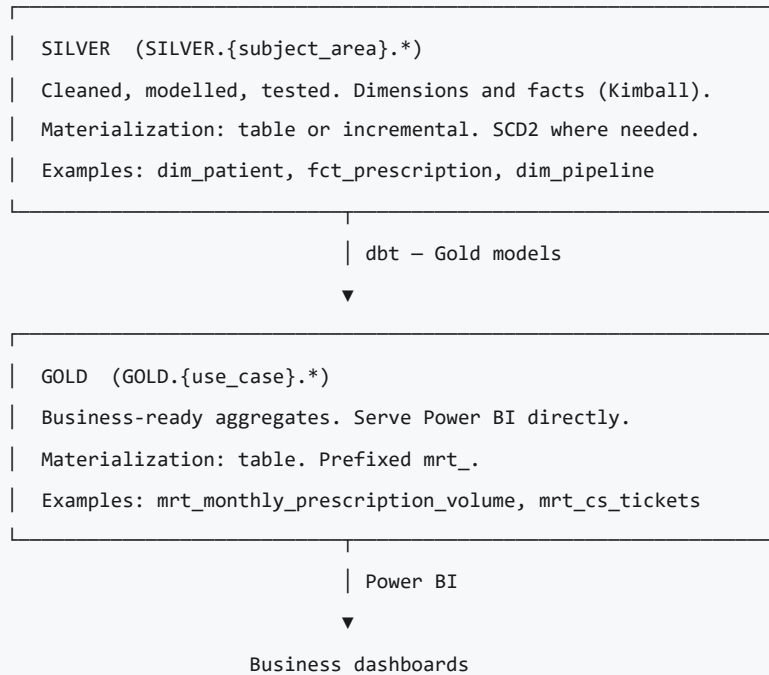
flowchart TD

```
SRC(["Source Systems<br/>HubSpot · Shopify · APIs"])
LAMBDA["AWS Lambda<br/>Ingestion"]
BRONZE["BRONZE<br/>raw · append-only<br/>Lambda owns – dbt does NOT touch"]
STAGING["STAGING<br/>views · stg_hubspot__*"]
SILVER["SILVER<br/>dim_* · fct_* · bridge_*"]
GOLD["GOLD<br/>mrt_*"]
PBI(["Power BI"])

SRC --> LAMBDA --> BRONZE
BRONZE -->|"source()"| STAGING
STAGING -->|"ref()"| SILVER
SILVER -->|"ref()"| GOLD
GOLD --> PBI
```

```
classDef dbt fill:#ecfdf5,stroke:#16a34a,color:#065f46
classDef lambda fill:#fef9c3,stroke:#ca8a04,color:#713f12
classDef bronze fill:#f1f5f9,stroke:#94a3b8,color:#475569
class STAGING,SILVER,GOLD dbt
class LAMBDA lambda
class BRONZE bronze
```





Layer ownership rules — non-negotiable:

Layer	Written by	Read by	dbt owns?
Bronze	Lambda / ingestion	dbt Staging	✗ No
Staging	dbt	dbt Silver	✓ Yes
Silver	dbt	dbt Gold, ad hoc analysis	✓ Yes
Gold	dbt	Power BI, business users	✓ Yes

dbt references Bronze as a **source**, not a `ref()`. Bronze tables are never built by dbt.

Part B — Declaring Sources in `sources.yml`

Before you can use `{{ source('hubspot', 'contacts') }}` in a model, you have to declare the source. This trips people up — dbt doesn't auto-discover tables. If the declaration is missing, the `source()` call fails at compile time.

```

# models/staging/sources.yml
version: 2

sources:
  - name: hubspot                                # the source alias used in {{ source() }}
    database: BRONZE                             # Snowflake database
    schema: HUBSPOT                              # Snowflake schema
    description: "HubSpot CRM data ingested via AWS Lambda."

    tables:
      - name: contacts

```

```
description: "One row per HubSpot contact. Append-only, deduplication happens in staging."
loaded_at_field: _ingested_at # column dbt uses for freshness checks

- name: deals
  description: "HubSpot deal records including pipeline stage history."
  loaded_at_field: _ingested_at

- name: pipeline_stages
  description: "Static lookup: pipeline stage definitions."
  # no loaded_at_field – static table, no freshness check needed
```

What dbt does with this:

- Registers `BRONZE.HUBSPOT.contacts` as a DAG node
- Enables `{{ source('hubspot', 'contacts') }}` to resolve correctly
- Enables `dbt source freshness` to check the `_ingested_at` column
- Shows the source in the DAG visualisation in `dbt docs serve`

Part C — `{{ source() }}` vs Hardcoding

Both of these produce the same SQL in the dev environment. They're not equivalent, though.

Hardcoded — never do this:

```
SELECT *
FROM BRONZE.HUBSPOT.contacts
```

With `source()` — always do this:

```
SELECT *
FROM {{ source('hubspot', 'contacts') }}
```

Here's what you lose by hardcoding:

	source()	Hardcoded
Shows in DAG	✔ Yes	✗ No
Freshness check works	✔ Yes	✗ No
Schema changes in one place	✔ Yes — update <code>sources.yml</code>	✗ No — update every model
Environment-aware	✔ Resolves based on <code>profiles.yml</code>	✗ Always points to prod
Detects if table deleted	✔ dbt warns	✗ Fails silently at runtime

Part D — Source Freshness

If HubSpot stops syncing data to Bronze, your Silver and Gold models will still run. They'll just be silently building on stale data. Source freshness gives you a way to catch that before it happens.

Source freshness checks whether Bronze data is up to date before dbt models run.

Declare freshness in `sources.yml` :

```
sources:
  - name: hubspot
    database: BRONZE
    schema: HUBSPOT

    freshness:                # applies to all tables unless overridden
      warn_after: {count: 6, period: hour}
      error_after: {count: 24, period: hour}

    tables:
      - name: contacts
        loaded_at_field: _ingested_at    # dbt queries MAX(_ingested_at) to check age

      - name: pipeline_stages
        freshness: null                 # static table – opt out of freshness check
```

Run a freshness check:

```
dbt source freshness
```

Output:

```
Found 1 source, 2 tables.
contacts: 3 hours 42 minutes ago – PASS
deals:    26 hours 15 minutes ago – ERROR
```

In your orchestrator: The freshness check runs before any dbt models. If a source errors, the pipeline stops. This prevents building Silver and Gold on stale Bronze data.

Exercise (25 min)

Project context: Three staging models exist from Module 04. The final staging model (`stg_hubspot__owners`) and its source declaration are missing. This session completes the staging layer.

Task 1 — Extend `sources.yml`

Open `models/staging/hubspot/sources.yml` . The file currently declares `contacts` and `deals` under the `hubspot` source block.

Add `owners` to the same source. The table lives at `BRONZE.HUBSPOT.owners` , is loaded every 12 hours via the `_ingested_at` column, and should warn after 14 hours and error after 25 hours.

► Entry to add

Task 2 — Write `stg_hubspot__owners.sql`

Create `models/staging/hubspot/stg_hubspot__owners.sql` .

The Bronze source (`BRONZE.HUBSPOT.owners`) has: `owner_id` , `first_name` , `last_name` , `email` , `_ingested_at` .

Requirements: view materialization; `{{ source('hubspot', 'owners') }}` ; all five columns; rename `_ingested_at` → `ingested_at` .

► Expected model

Task 3 — Compile and verify

```
dbt compile --select stg_hubspot__owners
```

Open the compiled output. Confirm it references `BRONZE.HUBSPOT.owners` — not a path you typed by hand.

Task 4 — Run all staging and check freshness

```
dbt run --select staging.*
dbt source freshness
```

Four staging models should run `OK` . The freshness report shows whether `owners` data is current. Read the output: which sources passed, warned, or errored?

Reference Material

- [dbt sources docs](#)
- [dbt source freshness](#)
- [dbt source\(\) function](#)
- Internal: `conventions` skill — schema naming, layer ownership

Prep Questions for Module 06

1. What must exist in `sources.yml` before you can use `{{ source('hubspot', 'contacts') }}` ?
2. Name two things you lose by hardcoding a Bronze table name instead of using `{{ source() }}` .
3. What column does dbt query to check source freshness?

4. Why does dbt NOT own the Bronze layer?